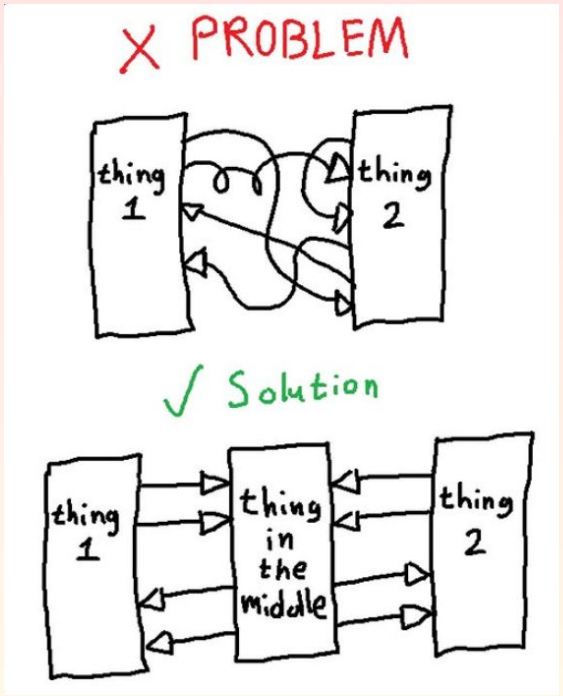
Game Engine Architectures 03

UI, Events && Best Practices

Important Design Principles

- We always aim for Loose Coupling, Separation of Concerns and Single-Responsibility
- There are many different ways for us to work towards them that UE provides us

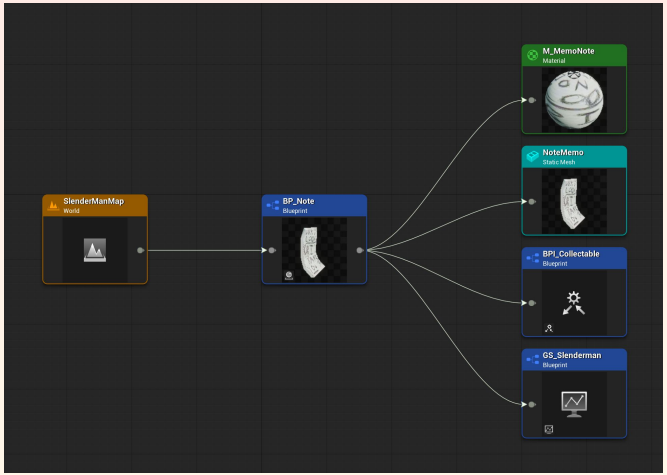
- **Loose Coupling:** components are only weakly associated and can work independently
- **Separation of Concerns:** complex systems should be separated into smaller parts
- **Single-Responsibility Principle:** one class should only have one responsibility or functionality



UE Reference Viewer

- If an Asset makes use of another Asset in some way, the first Asset references the second one
  > aka *Dependency*
- References are always unidirectional
- The Reference Viewer displays a graph of Assets that reference or are referenced by one or more Assets
- Select more than one Asset to see which references they have in common

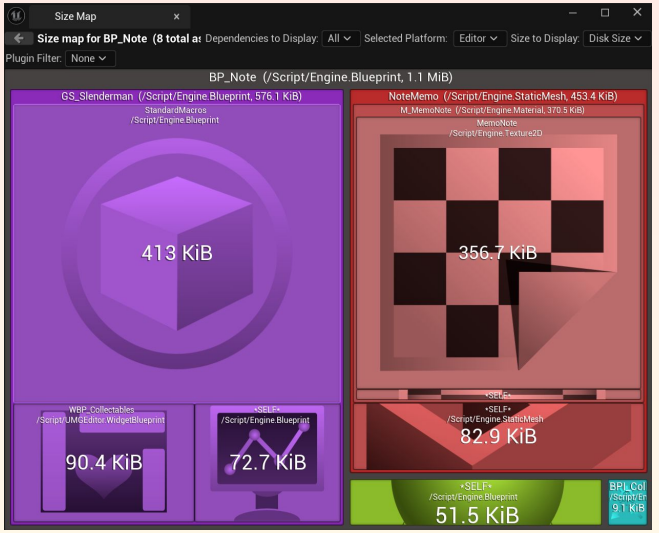
right click any asset in the project and click “Reference Viewer...” to show the asset’s references



UE Size Map

- Use the Size Map tool to view how much disk or memory space assets take up
- Every time an asset references another, its own size grows by the one it references (casting also counts!)
- This is often the reason people tell you to avoid references and casting; however, it's fine in some cases (e.g. the reference asset is very small, or you already have a reference to that asset somehow)

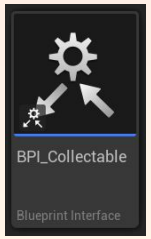
right click any asset in the project and click "Size Map..." to show the asset's size



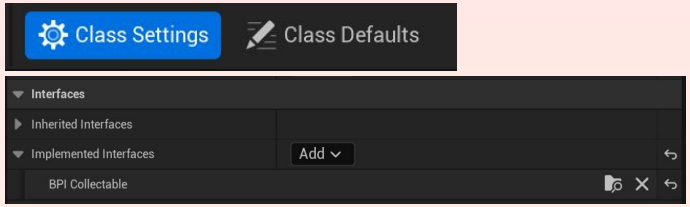
Now a few ways to solve this

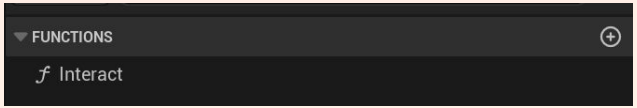
Blueprint Interfaces

- A way to define a common set of functions that different classes can implement
- These classes then share functionality, and other classes can interact with them
- It's like promising a specific mechanic
- Good for:
  - Enforcing Structure
  - Separation of Concerns
  - Decreasing Dependencies
  - Decreasing Memory Usage



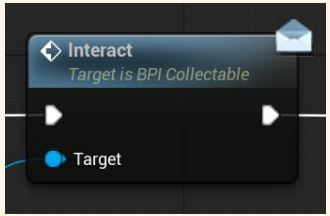
add in the Class Settings:





define a function and call from anywhere

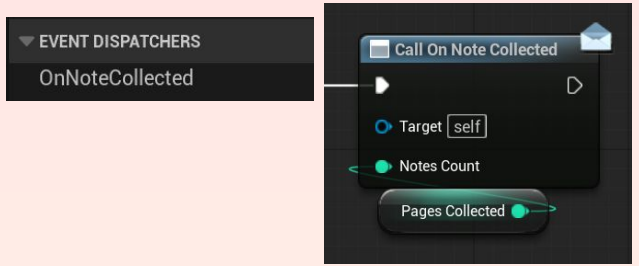
(don't forget to implement it in the class, though!)

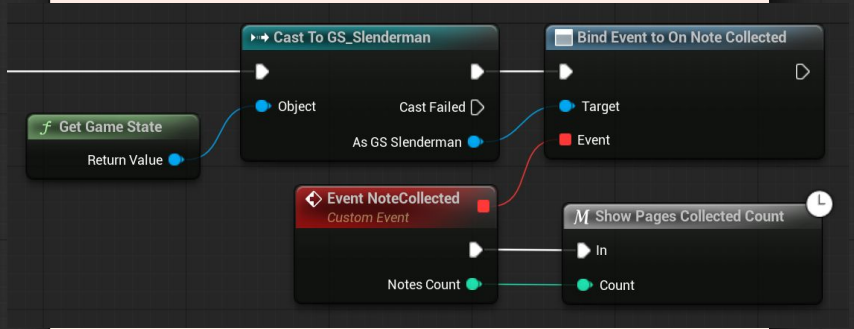


Event Dispatchers



- Events inside a BP can be bound to an *Event Dispatcher* (from the same or another BP)
- Once the dispatcher is called, all events bound to it will execute
- Good for:
  - Asynchronous Communication
  - Decoupling Classes
  - Modularity and Flexibility
- In C++: *Delegates*

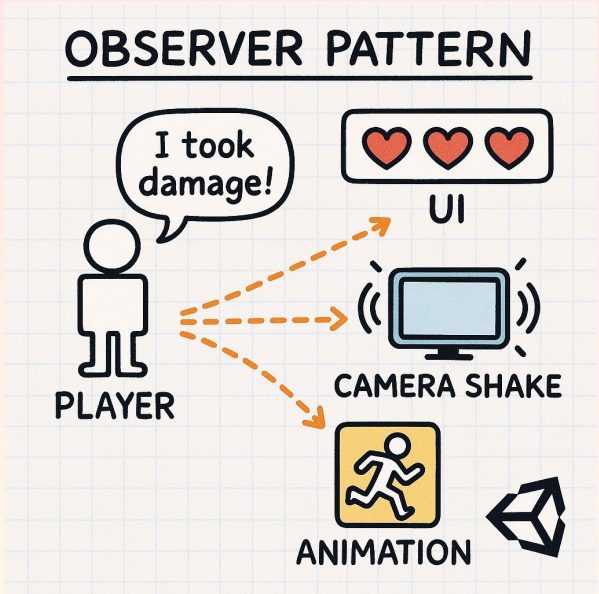




by binding it like this, we can also unbind at any time!

Observer Pattern

- One component says something, other components listen
- Creates a one-to-many relationship between a subject and its observers
- When the subject's state changes, all dependent observers are notified and updated automatically
- We use Event Dispatchers for this

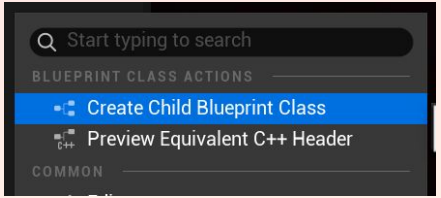


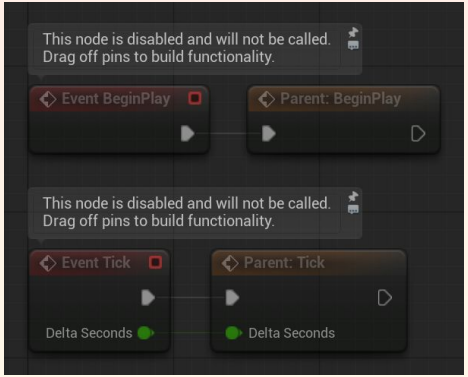
Inheritance



- To inherit the properties and methods of the parent class; Improves code by a lot!
- Enables code abstraction, reusability/scalability and reduces code duplication
- Children can freely access parent variables and functions
- Use for Polymorphism
- “Object” is the base class of all UE objects (Actors, Components etc. all inherit from it)

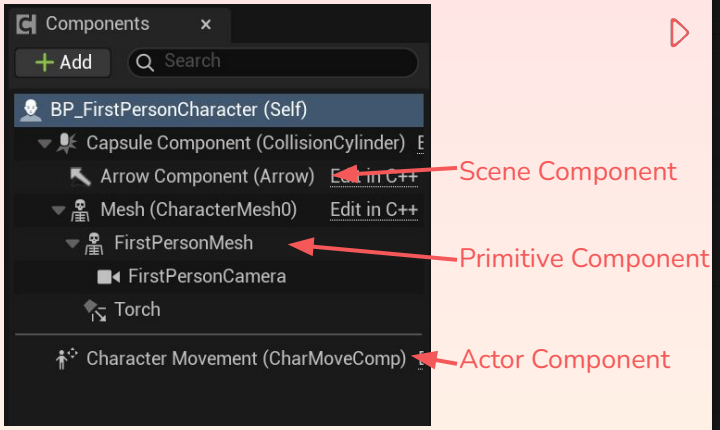
you can also right-click on any Project BP to create a child class:





Components

- A special type of Object that Actors can attach to themselves as sub-objects
- **Actor Components:** for abstract behaviours (e.g. movement, inventory management etc.). Don't have a transform
- **Scene Components:** have a transform but no geometric representation (e.g. cameras)
- **Primitive Components:** those are actually visible (Static and Skeletal meshes, Particle Systems etc.)

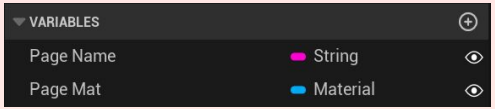


- You can create your own Actor Components
- Great for reusing mechanics!

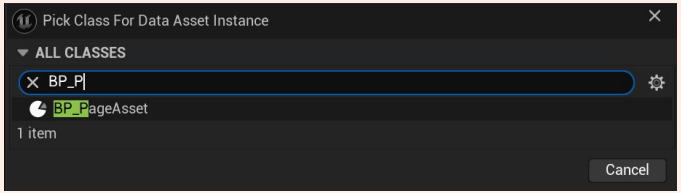
Data Assets

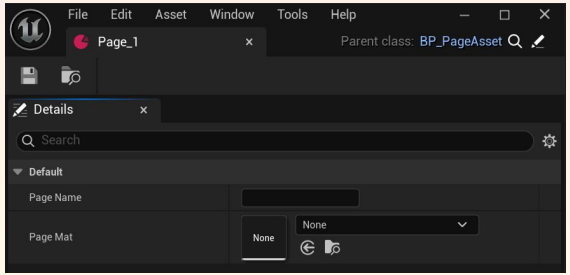
- Stores Data within its class instance
- Doesn't need to and can't be placed in the scene
- Similar to Scriptable Objects in Unity
- Can be referenced in other BPs
- UE automatically handles asset loading and unloading





Create *Data* > *Data Asset* and choose:





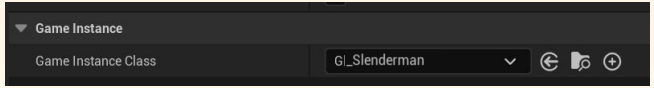
Some important Classes in UE

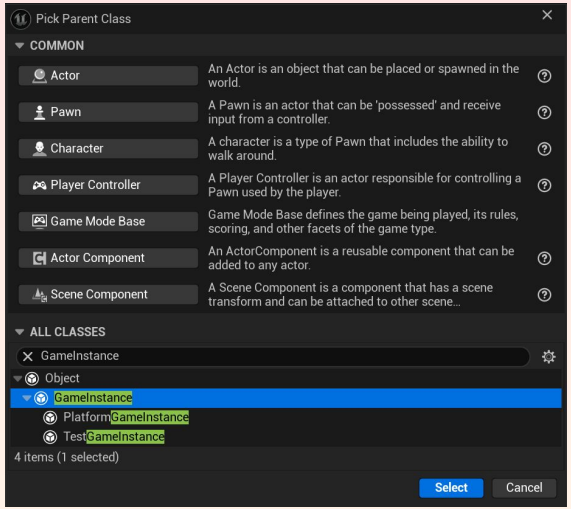
*you can’t do a lot wrong here when making Singleplayer games; these are mostly important for Multiplayer games

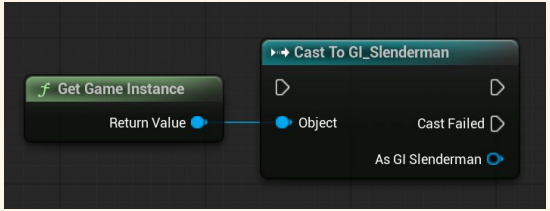
** you can add variables and logic in all of these classes

Game Instance

- Exists for the whole playtime, even when the level changes. An object spawned once you launch the game
- Used to manage persistent data and global game settings
- Can be accessed from anywhere
- Can be set in *Project Settings > Project > Maps & Modes > "Game Instance Class"*

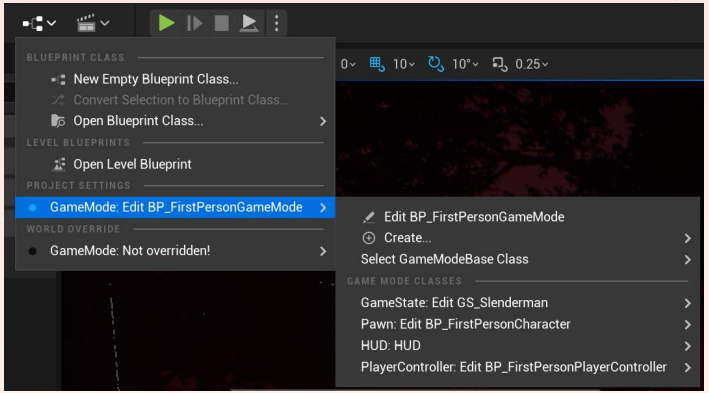






Game Mode

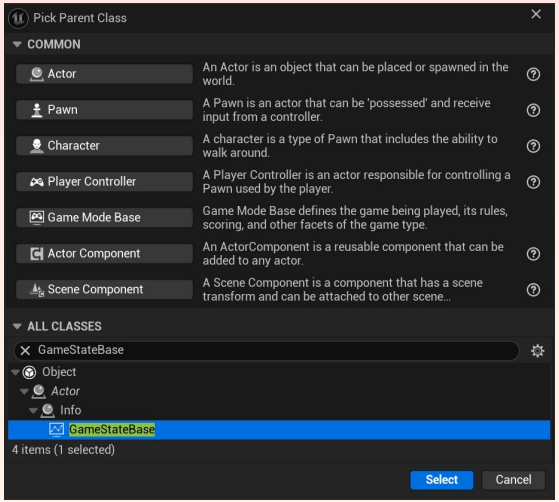
- The overall Game Manager - starts and stops the current game space you're in and handles the Game States
- Used for global initialisation and storage of level-dependent values
- Consists only for the current level! Spawned once the level is loaded
- Can also be set in *Project Settings > Project > Maps & Modes > "Default GameMode"* (which shows more settings)
- Can be accessed from anywhere



- Spawns all these (and more) classes on begin play
- Can be overridden per Level
- To create new, use Base Class "GameModeBase"

Game State

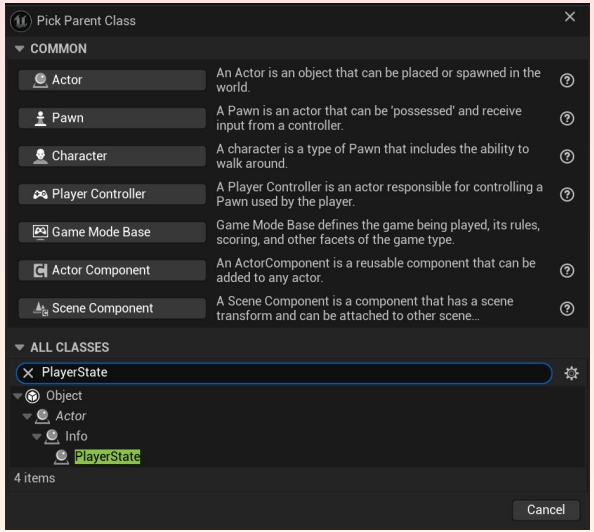
- Keeps track of every data relative to the current state of the game that all players in the game need to know about
- Examples: timers, scores, winning team, etc.
- You can change this at runtime to manage different states the game can be in (e.g. PregameState, IngameState, EndgameState)
- Spawned by the Game Mode. Can be accessed from anywhere

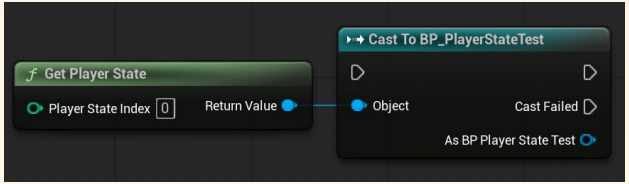




Player State

- Holds every variable not related to the PlayerCharacter that needs to be known by everyone
- Examples: player name, personal score, best score, etc.
- Spawned when a PlayerController is spawned (i.e. when a player arrives in the game)
- Can be accessed as a Variable from within the PlayerCharacter. Can also be accessed from anywhere with a specific player index

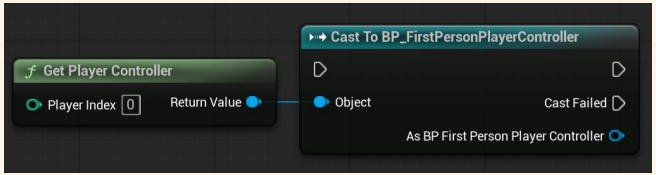




Player Controller && Player Character

Player Controller

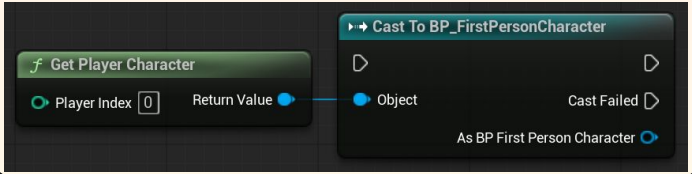
- Interface between Pawn and Player
- Used to set the active Input Mapping Context
- Persists throughout game while Pawn can change
- Player Input can be handled here or in Pawn
- Can be accessed from anywhere with a specific player index





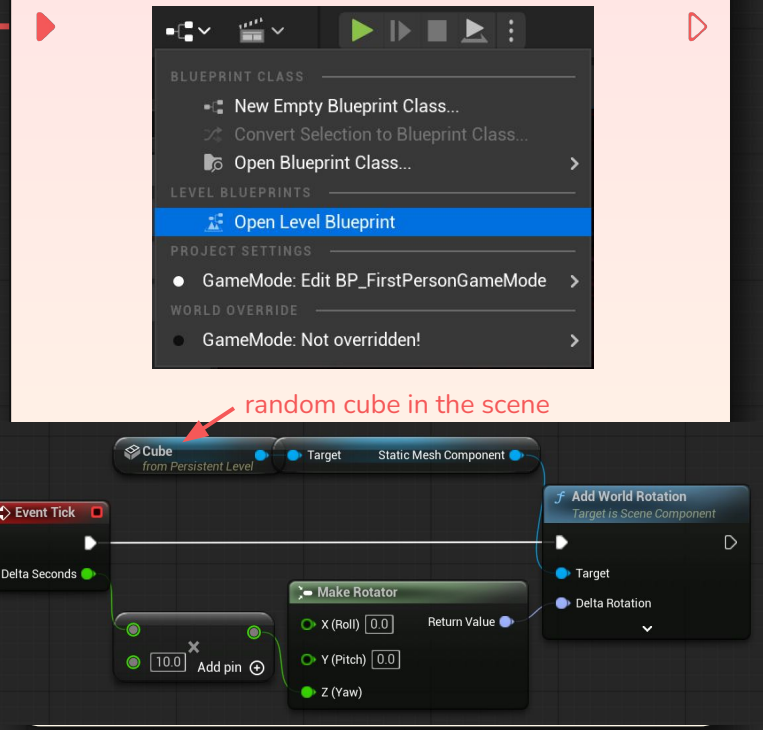
Player Character

- Executes actions in response to the Controller's input
- Holds personal info and stats; for example HP
- Is the Pawn Actor moved by the player (the actual visual thing we see in the scene)
- Can be accessed from anywhere with a specific player index



Level Blueprint

- Used for managing events and logic specific to a single level
- Allows to implement custom behaviour for object and environments within that level
- Unique to its level
- Can reference any actor or static mesh within the scene
- Don’t overuse!

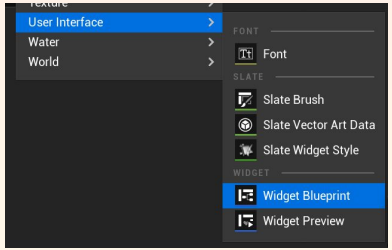


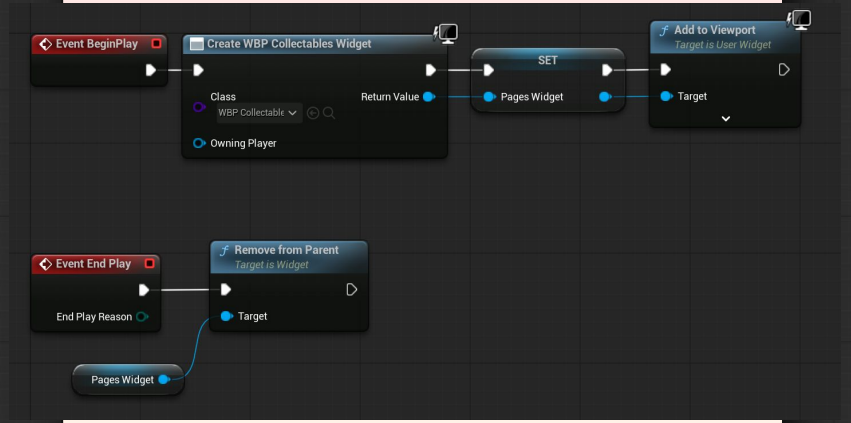
User Interface - Widgets

Widgets

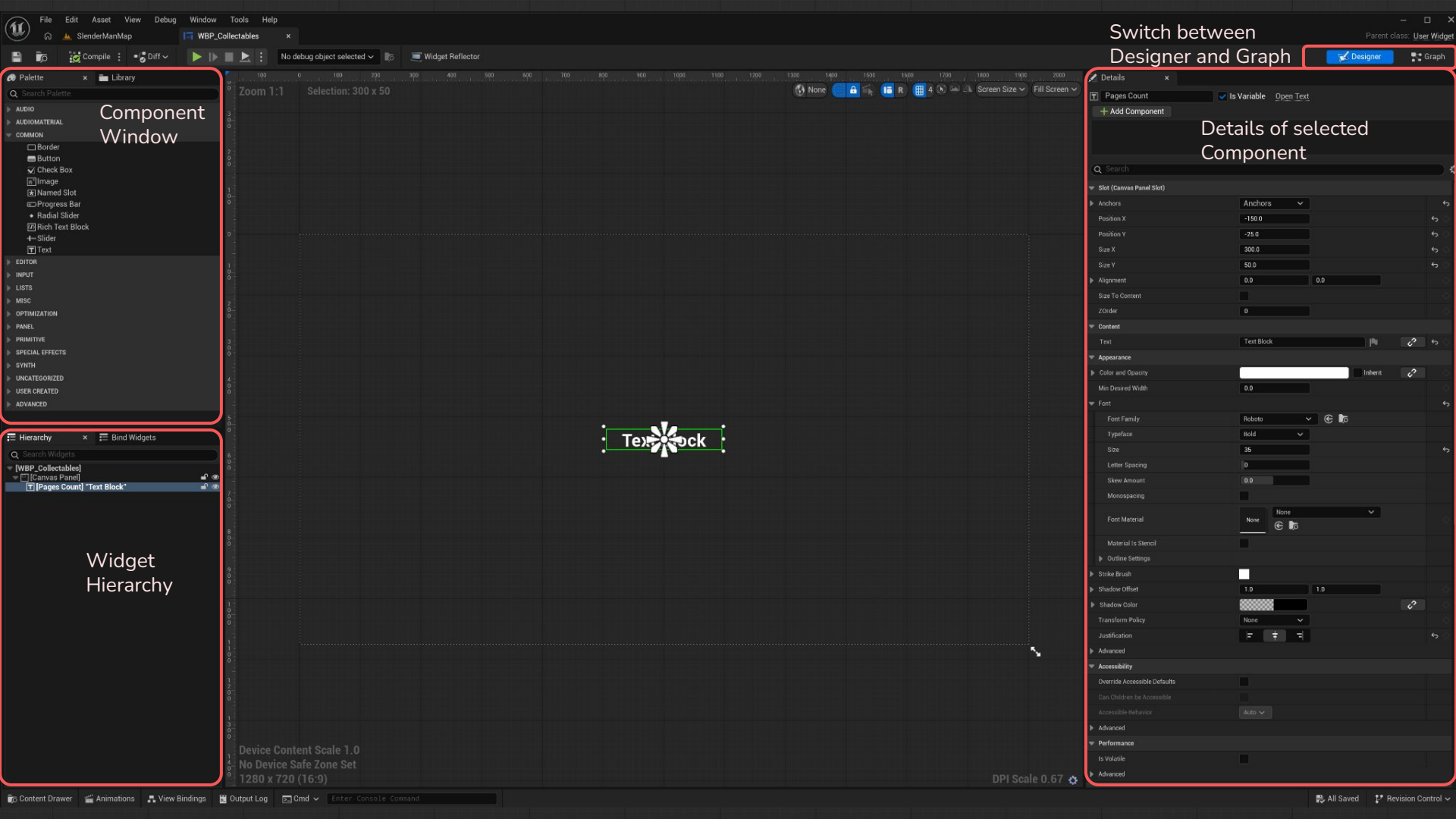


- Can be used as a pop-up, direct User-Interface or to display text, bars, stats... inside the scene
- Needs a Widget Blueprint
- Needs to be created and added to the viewport from within another BP in the game (e.g. Level BP or Character BP)





- save it in a variable to be able to easily remove it again at a later point


Switch between Designer and Graph
Component Window
Details of selected Component
Widget Hierarchy
Palette
Library
Hierarchy
Bind Widgets
Details
Designer
Graph
Zoom 1:1
Selection: 300 x 50
Device Content Scale 1.0
No Device Safe Zone Set
1280 x 720 (16:9)
DPI Scale 0.67